

Politecnico di Torino

Assignment 1 of the course

Information Theory for Data Science

Group Members:

Alessandro Conforti (s346511)

Alessandro Maini (s3346219)

Ali BehlooeiDolatsaraei (Student Number)

Sina Behnam (Student Number)

Hojjat Miryavifard (s334026)

October 8, 2024



Politecnico di Torino



In this report, we are going to solve the following exercises:

Exercises:

1. Entropy of a random variable with 3 outcomes (pt. 3)
2. Entropy of a random variable from a data series (pt. 4)
3. Application of the principle of maximum entropy (pt. 4)
4. Exercise on Information Gain Ratio (pt. 4)
5. Kullback-Leibler divergence from empirical distribution (pt. 3)
6. Permutation entropy for anomaly detection (pt. 5)
7. Information Gain Ratio and Classifier (pt. 7) (not yet available)



Table of Contents

- Exercise 1: Entropy of a Random Variable with 3 Outcomes
- Exercise 2: Entropy of a random variable from a data series
- Exercise3
- Exercise 4: Information Gain Ratio
- Exercise 5: Kullback-Leibler divergence from empirical distribution
- Exercise 6: Permutation Entropy Analysis
- Exercise 7: Information Gain Ratio and Classifiers



Entropy and Probability Vectors

In this exercise, we analyzed the entropy of a random variable with three possible outcomes. The entropy $H(p)$ is defined as:

$$H(p) = -\sum_{i=1}^{3} p_i \log_2 p_i$$

where $p = [p_1, p_2, p_3]$ represents the probability vector for the outcomes. The goal was to:

- Calculate entropy over the probability simplex.
- Apply an iterative averaging procedure to observe entropy evolution toward uniformity.
- Show that entropy approaches its maximum as the distribution becomes uniform.

Entropy over the Probability Simplex

We generated all valid probability vectors where $p_1 + p_2 + p_3 = 1$ and $p_i \geq 0$, and computed the entropy for each. The highest entropy occurs when the probabilities are equally distributed ($p_1 = p_2 = p_3$).

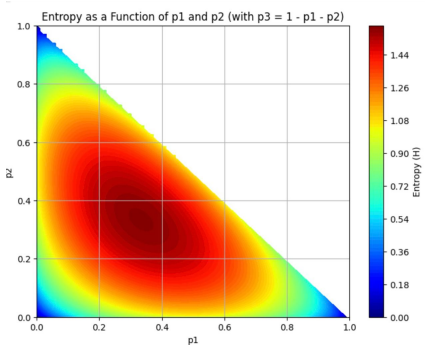


Figure: Figure 1: Entropy across the probability simplex. Highest entropy at uniform distribution.

Iterative Averaging Procedure

We started with a skewed probability vector $p = [0.98, 0.01, 0.01]$ and applied an iterative averaging procedure. Probabilities were updated and normalized after each iteration, demonstrating how the distribution evolves towards uniformity.

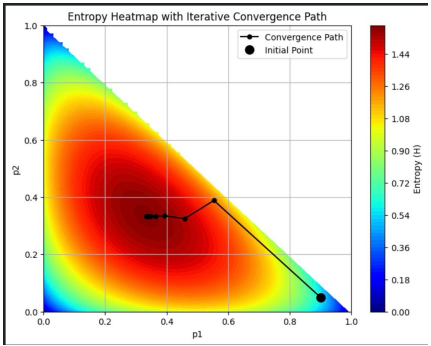


Figure: Figure 2: Entropy evolution during iterative averaging. Entropy rises as the distribution balances.



Results and Conclusion

In this exercise, we:

- Calculated entropy over the probability simplex, confirming that entropy is maximized when probabilities are uniformly distributed.
- Applied an iterative averaging process starting with a skewed probability vector. Each iteration brought the distribution closer to uniformity, with the corresponding entropy values increasing, approaching the theoretical maximum.
- Visualized the convergence of the entropy values as the probability vector approached uniformity.

Overall, the experiment demonstrated the direct relationship between entropy and distribution balance: skewed distributions have low entropy, while uniform distributions achieve maximum entropy. The iterative procedure successfully illustrated how entropy evolves as randomness in the distribution increases.



Table of Contents

- Exercise 1: Entropy of a Random Variable with 3 Outcomes
- Exercise 2: Entropy of a random variable from a data series
- Exercise3
- Exercise 4: Information Gain Ratio
- Exercise 5: Kullback-Leibler divergence from empirical distribution
- Exercise 6: Permutation Entropy Analysis
- Exercise 7: Information Gain Ratio and Classifiers



Entropy of a random variable from a data series

In this exercise, our aim is to:

- Identify a data series and estimate the probabilities of the outcomes based on their occurrences, updating the probabilities at each time step.
- At each time step, compute the entropy, plot its behaviour, and discuss the results

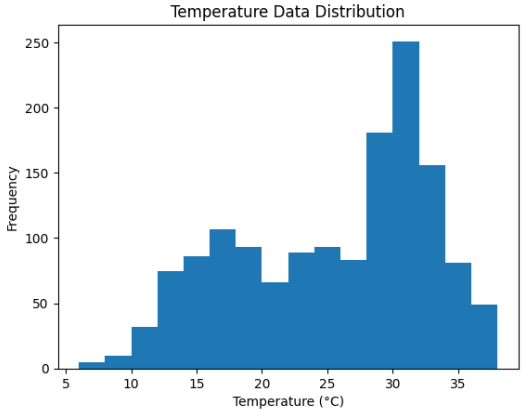


Data Series Overview

This dataset could be very suitable for our entropy calculation exercise, as it provides a real-world time series that likely exhibits interesting patterns and variations over time. The data is organized as a time series, meaning it's collected at regular time intervals (daily in this case) over a while. We have used Delhi's temperature record for over 4 years. This is the"link" to the dataset.
Although the dataset contains other information like date, mean temperature, humidity, wind speed, and mean pressure, we mainly focused on mean temperature and dropped all the rest.

Data Distribution

The distribution below represents temperature data distribution which shows the frequency of bins, each bar represents a 2°C temperature range, as specified in the code





Probability Estimation

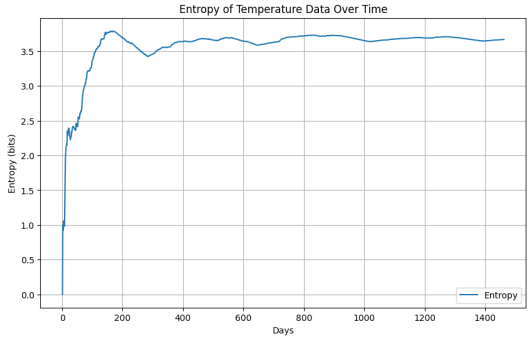
Binning is necessary to transform continuous temperature data into discrete categories, making it possible to compute the probabilities of the temperature falling within specific ranges. Once the data is binned, the frequency of data points in each bin is counted. These frequencies are then converted into probabilities by dividing each count by the total number of data points at each time step. These probabilities represent the likelihood of the temperature falling within each bin at a given time. using Shannon's entropy formula, entropy is computed for each time step based on the probabilities of the data falling within different bins.

Entropy $H(X)$ is calculated based on the following formula derived from Shannon's work:

$$H(X) = -\sum_{i=1}^{n} p(x_i) \log_2 p(x_i)$$

Iterative Calculation Over Time

The core of the code involves a loop that recalculates entropy as the dataset grows. At each iteration, the temperature data from day 1 to day ”i” (where i is the current time step) is considered.



Result

The process repeats, recalculating probabilities and entropy at every step, and the results are stored in a list called ”entropy values”. Finally, the entropy values are plotted against time (days). The x-axis represents the number of days (i.e., how much data has been accumulated), and the y-axis represents the entropy (measured in bits).
At the very beginning (first 50–200 days), the entropy rises sharply. This indicates that the uncertainty in the dataset is increasing rapidly. As more data is introduced, the system is becoming less predictable—likely because more variations in temperature are being captured. After the sharp rise, the entropy curve flattens out and oscillates around a value close to 3.5 bits. This stabilization indicates that the system has reached a point where additional data does not significantly increase uncertainty. In other words, the temperature data has become more predictable, and no new significant randomness is being introduced.



Result

The plot suggests that the temperature dataset initially had a lot of variability, but as more data was accumulated, the variability reached a limit. This pattern is common in systems where initial observations are more uncertain but then start to follow a more predictable pattern as the dataset grows.



Table of Contents

- Exercise 1: Entropy of a Random Variable with 3 Outcomes
- Exercise 2: Entropy of a random variable from a data series
- Exercise3
- Exercise 4: Information Gain Ratio
- Exercise 5: Kullback-Leibler divergence from empirical distribution
- Exercise 6: Permutation Entropy Analysis
- Exercise 7: Information Gain Ratio and Classifiers



Exercise 1: Title of the Exercise

Description

This is the content of Exercise 3.



Table of Contents

- Exercise 1: Entropy of a Random Variable with 3 Outcomes
- Exercise 2: Entropy of a random variable from a data series
- Exercise3
- Exercise 4: Information Gain Ratio
- Exercise 5: Kullback-Leibler divergence from empirical distribution
- Exercise 6: Permutation Entropy Analysis
- Exercise 7: Information Gain Ratio and Classifiers



Information Gain Ratio

In this exercise we aim to:

- Compute the Information Gain Ratio with respect to all the other variables and select the one that provides more information about X



DataSet

For This exercise we made a dataset which has some features like pageViews, Time on site, age group, and country and our target is purchase, we want to figure out which feature has the most impact on purchases.

Customer	Page Views	Time on Site	Age Group	Country	Purchase
1	1	12	18-25	USA	Yes
2	7	15	26-35	UK	No
3	8	8	18-25	Canada	No
4	4	12	36-45	USA	Yes
5	2	20	26-35	USA	Yes
6	3	60	36-45	UK	No
7	6	25	46-55	Australia	Yes
8	5	30	46-55	USA	No
9	15	35	26-35	Canada	Yes
10	2	3	18-25	USA	No



Calculate Information Gain

Our first step is calculate the Entropy of the target variable purchase (yes/no). This will serve as our baseline entropy for evaluating the effect of each feature.
The Entropy formula is:

$$H(X) = -\sum_{i=1}^{n} p(x_i) \log_2 p(x_i)$$

Then for each feature in the dataset calculates the Information Gain with respect to the target variable Purchases.
The Information Gain formula is:

$$IG(Y, X) = H(Y) - H(Y|X)$$

where:
$H(Y)$is the entropy of the target variable
$H(Y|X)$is the conditional entropy of the target variable given the feature X



Calculate Information Gain

In this exercise, we use the concept of Information Gain Ratio to determine the most influential factor among the given attributes for predicting a purchase.
Information gain(IG):Measures how much information each feature provides about the purchase, Higher values indicate more informative features and Range: from 0 (no information) to 1 (perfect information)
Gain Raito: Normalizes Information Gain by Split Information, Provides a fairer comparison between features, Higher values indicate better predictive power



Calculation Information Gain

The next step is to calculate the conditional entropy of the target variable given each of the other features. This measures the remaining uncertainty about the target variable after knowing the value of a feature.
For each feature (like time on site), the conditional entropy is calculated as:

$$H(Purchase|TimeOnSite) = -\sum_{i=1}^{n} p(x_i).H(purchase|x_i)$$

where $H(Purchase|X_i)$ is the entropy of the target variable within each group of the feature.



Calculate Information Gain

The formula for Information Gain Ratio is:

$$IGR(Y,X) = \frac{IG(Y,X)}{H(X)}$$

After calculating the Information Gain Ratio for each feature, selected the feature with the highest value.

```
Information Gain for Page Views: 0.8000
Information Gain for Time on Site: 1.0000
Information Gain for Age Group: 0.0490
Information Gain for Country: 0.3145


Gain Ratio for Page Views: 0.2563
Gain Ratio for Time on Site: 0.3203
Gain Ratio for Age Group: 0.0249
Gain Ratio for Country: 0.1786



The feature with the highest Gain Ratio is: Time on Site
```



Result

The information gain measures how much knowing a feature reduces uncertainty about the target variable. A high information gain means the feature provides significant information about the target.we compute The Information Gain Ratio which adjusts the information gain by accounting for the intrinsic information of the feature itself. Features with more distinct values (like those with more categories) tend to have higher information gain, so the Information Gain Ratio helps to normalize this bias.Finally we can the feature which provides the most important information for target variable



Table of Contents

- Exercise 1: Entropy of a Random Variable with 3 Outcomes
- Exercise 2: Entropy of a random variable from a data series
- Exercise3
- Exercise 4: Information Gain Ratio
- Exercise 5: Kullback-Leibler divergence from empirical distribution
- Exercise 6: Permutation Entropy Analysis
- Exercise 7: Information Gain Ratio and Classifiers



Kullback-Leibler divergence

In this exercise we try to identify which binomial distribution is closest to our sampled distribution.

To do so, we need to introduce the concept of **Kullback-Leibler divergence**.

Kullback-Leibler divergence is a statistical measure that quantifies how much one probability distribution differs from a second, reference probability distribution. KL divergence is closely linked to conditional entropy and information gain, because it measures how much more uncertainty is introduced by using one probability distribution instead of another.

For two probability distributions P (the observed distribution) and Q (the analytical fit), the KL divergence from Q to P is given by:

$$D_{KL}(P||Q) = \sum_x P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$



Observed Distribution

Consider the following observed data:

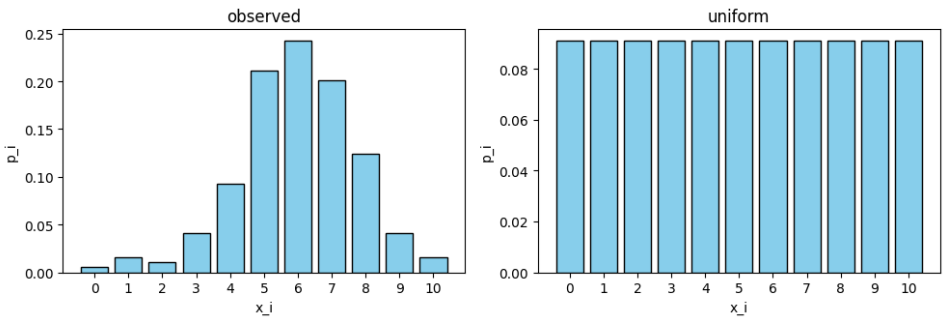
$$x_i = (0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad 7 \quad 8 \quad 9 \quad 10)$$

$$\#\ \ observed \quad x_i = (1 \quad 3 \quad 2 \quad 8 \quad 18 \quad 41 \quad 47 \quad 39 \quad 24 \quad 8 \quad 3)$$

Divergence from Uniform Distribution

We compute the KL divergence of the observed distribution from a uniform distribution, to measure how distant the two probability distributions are.

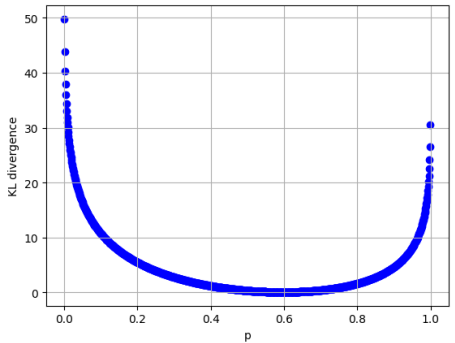
KL divergence: 0.6608771259419962





Search of the optimal Binomial Distribution

To find the binomial distribution closest to the observed distribution, we compute the KL divergence for each binomial distribution given $n = 11$ independent experiments and probability of success p ranging between 0 and 1 (step = 0.001).



Best Binomial Distribution

From the previous calculations we obtain that the binomial distribution that minimizes the KL divergence has $p = 0.596$. We must therefore choose this p-value if we want to approximate the observed distribution with a binomial distribution, thus minimizing the error committed.

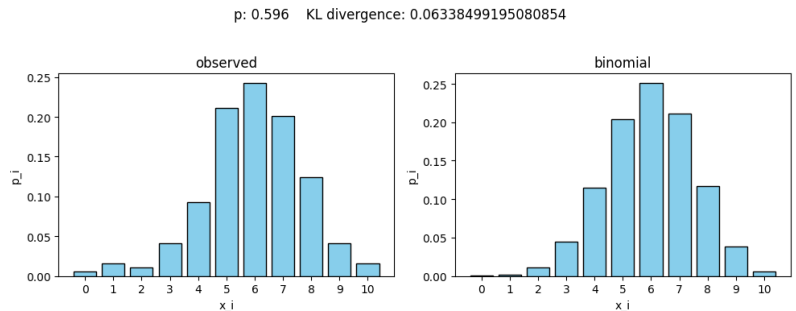




Table of Contents

- Exercise 1: Entropy of a Random Variable with 3 Outcomes
- Exercise 2: Entropy of a random variable from a data series
- Exercise3
- Exercise 4: Information Gain Ratio
- Exercise 5: Kullback-Leibler divergence from empirical distribution
- Exercise 6: Permutation Entropy Analysis
- Exercise 7: Information Gain Ratio and Classifiers



Data Generation and Parameters

We generated a random data series of length 10,000 using a uniform distribution between 0 and 1:

$$x = [x_1, x_2, \ldots, x_{10000}], \quad x_i \in [0, 1]$$

The entropy calculation was carried out using:

- Sliding window size: $N_w = 512$
- Permutation order: $m = 3$



Sliding Window and Permutation Entropy

The data series was divided into overlapping windows of size $N_w = 512$. Within each window, we mapped the sequence to a permutation by sorting the indices of the data values.

The permutation entropy for each window was calculated as:

$$H(\pi) = -\sum P(\pi) \log_2 P(\pi), \quad P(\pi) = \frac{\text{count of permutation } \pi}{\text{total number of permutations in the window}}$$

The entropy was then normalized:

$$H' = \frac{H(\pi)}{\log_2(m!)}$$

where H' ranges from 0 (fully predictable data) to 1 (completely random data).

Random Data Series Plot

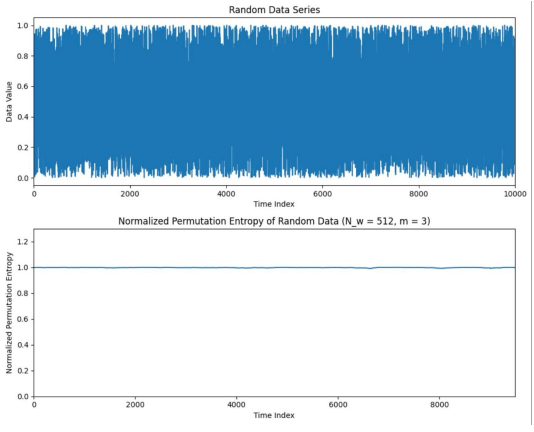


Figure: Random data series and corresponding permutation entropy, showing high randomness with entropy near the maximum value.



Data Modification: Structured Pattern

To introduce a structured pattern, the segment between indices **3000** and **4000** was partially sorted in chunks of size 20:

$$x_{i:i+20} = \text{sort}(x_{i:i+20}) \quad \text{for } i \in [3000, 4000]$$

This reduced the randomness while keeping the mean and variance of the data consistent.

Modified Data Series Plot

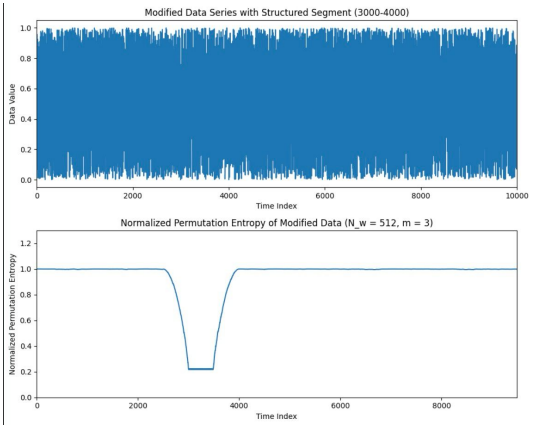


Figure: Modified data series with a structured segment (indices 3000 to 4000). The entropy decreases, indicating reduced randomness.



Results and Analysis

For the **random data series**, the entropy remained close to the theoretical maximum:

$$H_{\max} = \log_2(m!) = \log_2(6) \approx 2.585$$

The normalized permutation entropy approached **1**, implying a high degree of randomness.
After modifying the segment between indices **3000** and **4000**, the entropy dropped due to the partial sorting, indicating reduced randomness in that region:

$$H' < 1 \quad \text{for the modified region}$$



Conclusion

The **permutation entropy** effectively captured changes in the time series complexity. The random data exhibited entropy near the theoretical maximum, while the structured segment showed a decrease in entropy, indicating the presence of a pattern. Permutation entropy is useful for detecting complexity changes and can be applied in anomaly detection in more complex datasets.



Table of Contents

- Exercise 1: Entropy of a Random Variable with 3 Outcomes
- Exercise 2: Entropy of a random variable from a data series
- Exercise3
- Exercise 4: Information Gain Ratio
- Exercise 5: Kullback-Leibler divergence from empirical distribution
- Exercise 6: Permutation Entropy Analysis
- Exercise 7: Information Gain Ratio and Classifiers



Information Gain Ratio and Classifiers

In this exercise we build a decision tree classifier using the principle of maximum information gain ratio.
A decision tree classifier is an algorithm that classifies a vector of feaures analyzing a single feature in each node and then outputting a class when it reaches a leaf node
These kind of algorithms are built using a training set of vector, whose classes are known, in a way that they classify the vectors of the training set as right as possibile, even if it's not possibile to guarantee that every vector in the training set will be classified correctly
Each categorical feaures is analysed only once in the tree, but numerical feautures can be analysed multiple times, using different threshold for comparison



Selecting features

When building a decision tree classifier, we must choose a feature to analyse at each node and, in the case of numerical features, also a threshold to use for comparison
Using the information gain ratio, we made the choice in order to gain the maximum amount of information about the resulting class at each step
The class and the features are treated like random variables, whose outcomes have probabilities calculated considering the occurrencies of the values in the training data set
The information gain ratio is calculated using this formula

$$IG(C|X_n) = \frac{H(C) - H(C|X_n)}{H(X_n)}$$

The value is normalized with respect to the entropy of the feature, to failry compare features with different number of outcomes



Building a decision tree

To select the feature to analyze at each node, we follow this iterative process

- If all the vector in the training set belong to the same class, that is a leaf and the value of that class is returned
- If no more feature can be analyzed and the vectors of the training set belong to different classes, return the class which is more common among the training vectors
- We calculate the normalized information gain of the class relative to each feature and, in the case of numerical features, for each possibile threshold
- We select the feature, and eventually the threshold, that have the highest information gain ratio
- We create branches for each value of the feature chosen, or for each result of the comparison of the feature with the threshold. To each branch is assigned a split of the training set made by all the vector whose values of the feature satisfy the subtree condition and we repeat the process for each subtree



Training set

We build our classifier using the provided training set. The first and third features are numerical, while the second one can only assume the value of 0 and 1

x_1	x_2	x_3	C
20	0	10	0
20	0	70	0
20	1	20	1
20	1	80	0
40	0	40	1
40	1	60	1
40	1	50	1
20	0	60	0
60	1	60	1



Class and features entropy

The first thing we do is calculate the entropy of the class and the entropy of every feature on the whole training set

$$H(C) = -\frac{4}{9} * log2\left(\frac{4}{9}\right) - \frac{5}{9} * log2\left(\frac{5}{9}\right) = 0.9911$$

$$H(x_1) = -\frac{5}{9} * log2\left(\frac{5}{9}\right) - \frac{3}{9} * log2\left(\frac{3}{9}\right) - \frac{1}{9} * log2\left(\frac{1}{9}\right) = 1.3516$$

$$H(x_2) = -\frac{4}{9} * log2\left(\frac{4}{9}\right) - \frac{5}{9} * log2\left(\frac{5}{9}\right) = 0.9911$$

$$H(x_3) = -6 * \frac{1}{9} * log2\left(\frac{1}{9}\right) - \frac{3}{9} * log2\left(\frac{3}{9}\right) = 2.6416$$



First feature information gain ratio

The first feature has two thresholds: 30 and 50
The information gain ratio for the threshold 30 is

$$H(C|x_1 < 30) = -\frac{4}{5} * log2\left(\frac{4}{5}\right) - \frac{1}{5} * log2\left(\frac{1}{5}\right) = 0.7219$$

$$H(C|x_1 > 30) = 0$$

$$H(C|x_1) = \frac{5}{9} * H(C|x_1 < 30) + \frac{4}{9} * H(C|x_1 > 30) = 0.4011$$

$$Ig(C|x_1) = \frac{H(C) - H(C|x_1)}{H(x_1)} = 0.4365$$



First feature information gain ratio

The information gain ratio for the threshold 50 is

$$H(C|x_1 < 50) = -\frac{4}{8} * log2\left(\frac{4}{8}\right) - \frac{4}{8} * log2\left(\frac{4}{8}\right) = 1$$

$$H(C|x_1 > 50) = 0$$

$$H(C|x_1) = \frac{8}{9} * H(C|x_1 < 50) + \frac{1}{9} * H(C|x_1 > 50) = \frac{8}{9} = 0.8889$$

$$Ig(C|x_1) = \frac{H(C) - H(C|x_1)}{H(x_1)} = 0.0756$$



Second feature information gain ratio

The information gain ratio for the second feature is

$$H(C|x_2 = 0) = -\frac{3}{4} * log2\left(\frac{3}{4}\right) - \frac{1}{4} * log2\left(\frac{1}{4}\right) = 0.8113$$

$$H(C|x_2 = 1) = -\frac{4}{5} * log2\left(\frac{4}{5}\right) - \frac{1}{5} * log2\left(\frac{1}{5}\right) = 0.7219$$

$$H(C|x_2) = \frac{4}{9} * H(C|x_2 = 0) + \frac{5}{9} * H(C|x_2 = 1) = 0.7616$$

$$Ig(C|x_2) = \frac{H(C) - H(C|x_2)}{H(x_2)} = 0.2315$$



Third feature information gain ratio

The third feature has 6 thresholds: 15, 30, 45, 55, 65 and 75
The information gain ratio for the threshold 15 is

$$H(C|x_3 < 15) = 0$$

$$H(C|x_3 > 15) = -\frac{3}{8} * log2\left(\frac{3}{8}\right) - \frac{5}{8} * log2\left(\frac{5}{8}\right) = 0.9544$$

$$H(C|x_3) = \frac{1}{9} * H(C|x_3 < 15) + \frac{8}{9} * H(C|x_3 > 15) = 0.8484$$

$$Ig(C|x_3) = \frac{H(C) - H(C|x_3)}{H(x_3)} = 0.0540$$



Third feature information gain ratio

The information gain ratio for the threshold 30 is

$$H(C|x_3 < 30) = -log2\left(\frac{1}{2}\right) = 1$$

$$H(C|x_3 > 30) = -\frac{3}{7} * log2\left(\frac{3}{7}\right) - \frac{4}{7} * log2\left(\frac{4}{7}\right) = 0.9852$$

$$H(C|x_3) = \frac{2}{9} * H(C|x_3 < 30) + \frac{7}{9} * H(C|x_3 > 30) = 0.9885$$

$$Ig(C|x_3) = \frac{H(C) - H(C|x_3)}{H(x_3)} = 9.711 * 10^{-4}$$



Third feature information gain ratio

The information gain ratio for the threshold 45 is

$$H(C|x_3 < 45) = -\frac{1}{3} * log2\left(\frac{1}{3}\right) - \frac{2}{3} * log2\left(\frac{2}{3}\right) = 0.9183$$

$$H(C|x_3 > 45) = -\frac{3}{6} * log2\left(\frac{3}{6}\right) - \frac{3}{6} * log2\left(\frac{3}{6}\right) = 1$$

$$H(C|x_3) = \frac{3}{9} * H(C|x_3 < 45) + \frac{6}{9} * H(C|x_3 > 45) = 0.9728$$

$$Ig(C|x_3) = \frac{H(C) - H(C|x_3)}{H(x_3)} = 0.0069$$



Third feature information gain ratio

The information gain ratio for the threshold 55 is

$$H(C|x_3 < 55) = -\frac{3}{4} * log2\left(\frac{3}{4}\right) - \frac{1}{4} * log2\left(\frac{1}{4}\right) = 0.8133$$

$$H(C|x_3 > 55) = -\frac{3}{5} * log2\left(\frac{3}{5}\right) - \frac{2}{5} * log2\left(\frac{2}{5}\right) = 0.9710$$

$$H(C|x_3) = \frac{4}{9} * H(C|x_3 < 45) + \frac{5}{9} * H(C|x_3 > 45) = 0.9$$

$$Ig(C|x_3) = \frac{H(C) - H(C|x_3)}{H(x_3)} = 0.0345$$



Third feature information gain ratio

The information gain ratio for the threshold 65 is

$$H(C|x_3 < 65) = -\frac{2}{7} * log2\left(\frac{2}{7}\right) - \frac{5}{7} * log2\left(\frac{5}{7}\right) = 0.8631$$

$$H(C|x_3 > 65) = 0$$

$$H(C|x_3) = \frac{7}{9} * H(C|x_3 < 65) + \frac{2}{9} * H(C|x_3 > 65) = 0.6713$$

$$Ig(C|x_3) = \frac{H(C) - H(C|x_3)}{H(x_3)} = 0.1210$$



Third feature information gain ratio

The information gain ratio for the threshold 75 is

$$H(C|x_3 < 75) = -\frac{3}{8} * log2\left(\frac{3}{8}\right) - \frac{5}{8} * log2\left(\frac{5}{8}\right) = 0.9544$$

$$H(C|x_3 > 75) = 0$$

$$H(C|x_3) = \frac{8}{9} * H(C|x_3 < 75) + \frac{1}{9} * H(C|x_3 > 75) = 0.8484$$

$$Ig(C|x_3) = \frac{H(C) - H(C|x_3)}{H(x_3)} = 0.0540$$



First node feature choice

The highest information gain ratio is the one of the first feature with threshold 30, so that will be the first node of the classifier

All the vector of the training set split on the right part of the node belog to the same class, so that is a leaf and the class 1 is returned

We now apply the iterative procedure to the left split of the training set

x_1	x_2	x_3	C
20	0	10	0
20	0	70	0
20	1	20	1
20	1	80	0
20	0	60	0



Class and features entropy

We calculate the class and features entropy on the new training set

$$H(C) = -\frac{4}{5} * log2\left(\frac{4}{5}\right) - \frac{1}{5} * log2\left(\frac{1}{5}\right) = 0.7219$$

$$H(x_1) = 0$$

$$H(x_2) = -\frac{3}{5} * log2\left(\frac{3}{5}\right) - \frac{2}{5} * log2\left(\frac{2}{5}\right) = 0.9710$$

$$H(x_3) = -log2\left(\frac{1}{5}\right) = 2.3219$$



Second feature information gain ratio

$$H(C|x_2 = 0) = 0$$

$$H(C|x_2 = 1) = -log2\left(\frac{1}{2}\right) = 1$$

$$H(C|x_2) = \frac{3}{5} * H(C|x_2 = 0) + \frac{2}{5} * H(C|x_2 = 1) = 0.4$$

$$Ig(C|x_2) = \frac{H(C) - H(C|x_2)}{H(x_2)} = 0.3316$$



Third feature information gain ratio

The thrid feature has four thresholds: 15, 40, 65, 75
The information gain ratio for the threshold 15 is

$$H(C|x_3 < 15) = 0$$

$$H(C|x_3 > 15) = -\frac{3}{4} * log2\left(\frac{3}{4}\right) - \frac{1}{4} * log2\left(\frac{1}{4}\right) = 0.8113$$

$$H(C|x_3) = \frac{1}{5} * H(C|x_3 < 15) + \frac{4}{5} * H(C|x_3 > 15) = 0.6490$$

$$Ig(C|x_3) = \frac{H(C) - H(C|x_2)}{H(x_2)} = 0.0314$$



Third feature information gain ratio

The information gain ratio for the threshold 40 is

$$H(C|x_3 < 40) = -log2\left(\frac{1}{2}\right) = 1$$

$$H(C|x_3 > 40) = 0$$

$$H(C|x_3) = \frac{2}{5} * H(C|x_3 < 40) + \frac{3}{5} * H(C|x_3 > 40) = 0.4$$

$$Ig(C|x_3) = \frac{H(C) - H(C|x_2)}{H(x_2)} = 0.1386$$



Third feature information gain ratio

The information gain ratio for the threshold 65 is

$$H(C|x_3 < 65) = -\frac{2}{3} * log2\left(\frac{2}{3}\right) - \frac{1}{3} * log2\left(\frac{1}{3}\right) = 0.9183$$

$$H(C|x_3 > 65) = 0$$

$$H(C|x_3) = \frac{3}{5} * H(C|x_3 < 65) + \frac{2}{5} * H(C|x_3 > 65) = 0.5510$$

$$Ig(C|x_3) = \frac{H(C) - H(C|x_2)}{H(x_2)} = 0.0736$$



Third feature information gain ratio

The information gain ratio for the threshold 75 is

$$H(C|x_3 < 75) = -\frac{3}{4} * log2\left(\frac{3}{4}\right) - \frac{1}{4} * log2\left(\frac{1}{4}\right) = 0.8113$$

$$H(C|x_3 > 75) = 0$$

$$H(C|x_3) = \frac{4}{5} * H(C|x_3 < 75) + \frac{1}{5} * H(C|x_3 > 75) = 0.6490$$

$$Ig(C|x_3) = \frac{H(C) - H(C|x_2)}{H(x_2)} = 0.0314$$

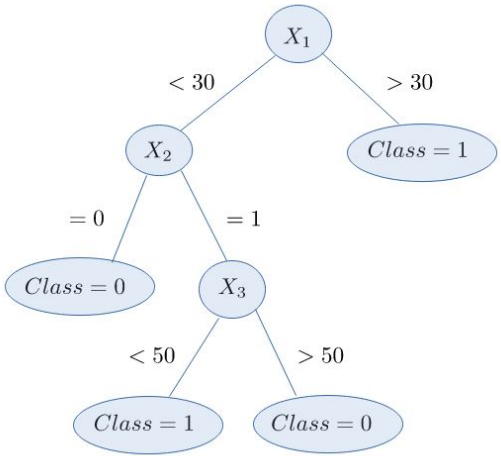


Second and thrid nodes feature choice

The highest information gain ratio is the one of the second feature, so that will be the second node of the classifier. All the vector on the left side of the tree belog to the same class, so that is a leaf and the class 0 is returned
On the right side there are just two vector, which belong to two different classes, whose only differing feature is the third one, so that is the only possible choice. The threshold used is 50, the average of the two feature values
The resulting two nodes have only one vecor each, so they are leaf and the class of the vector is returned

Classifier plot

This is the plot of the resulting classifier





Vectors classification

The classifier is implemented in the Classifier.m matlab code file. It correctly classifies each vector of the training set. This follows from the fact that for each leaf all the training vector of that leaf belong to the same class, so there are no leaf that return values according to the majority rule, which would result in certain error for at least one training set vector. The classifier can be used to classify arbitrary vector that don't belong to the training set. Here are some example of classfication of random vectors

x_1	x_2	x_3	C
10	0	75	0
35	1	90	1
15	1	45	1
5	1	75	0



Confusion matrix

The confusion matrix of a classifier is a matrix that contains information about how good is the classifier and how many error it makes, which is why is also called error matrix
Each row of the matrix represent a real value of the class of a vector, while each coloumn represent a predicted value of the class of a vector
The (i,j) value of the matrix is the number of vector of class i that are being classified as class j by the classifier. It follows that the elements on the diagonal of the confusion matrix are the number of vectors that have been classified correctly, for each possibile value of the class, while the others are the vector that have been classified incorrectly
The confusion matrix of our decision tree classifier is

$$\begin{bmatrix} 4 & 0 \\ 0 & 5 \end{bmatrix}$$



Politecnico di Torino *Thank you for listening!*

Any questions?